

Lesson 7

Installing and Configuring an OS

Topic 7D

Use Scripts to Configure Servers

Exam Objectives Covered

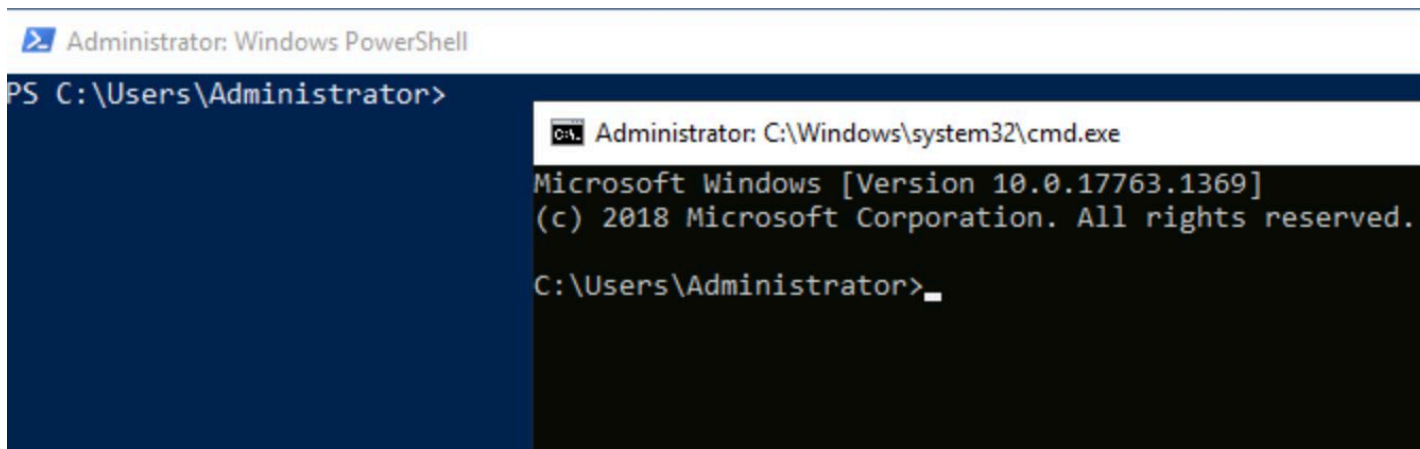
- 2.1 Given a scenario, install server operating systems
- 2.6 Summarize scripting basics for server administration

What are Scripts?

- Advantages of scripts
 - Consistent
 - Fast
 - Scheduled

Shell Languages

- Batch file
- Bash
- PowerShell
- VBS



PowerShell/cmd.exe. (Screenshot courtesy of Microsoft.)

Comment Syntax

```
1 #!/bin/bash
```

```
2
```

Script with commented lines.

```
3 # This script displays the string "Hello World" on the screen.
```

```
4 echo "Hello World"
```

Environment Variables

```
[admin@localhost ~]$ echo $HOSTNAME
fedora
[admin@localhost ~]$ echo $USER
admin
[admin@localhost ~]$ echo $HOME
/home/admin
[admin@localhost ~]$ _
```

Displaying variables.

Basic Script Constructs

- Loops
- Conditionals
- Comparators

Functions

- Discrete sections of code within a script
- Appears at the start of the script

Basic Data Types

- Integers

```
[admin@localhost ~]$ echo $((2+2))  
4  
[admin@localhost ~]$ █
```

echo \$((2+2)) value is summed and displayed.

- Strings
- Arrays

Server Administration Scripting Tasks

- Scripts
 - Login and logout
 - Startup and shutdown
 - Account management
 - Service
 - Bootstrap

Guidelines for Using Scripts to Configure Servers

- Script repetitive tasks
- Schedule for off-hours
- PowerShell for Windows
- Bash for Linux
- Use comments liberally
- Commented lines start with the # character

Review Activity: Configuring Servers with Scripts



Review Activity

Introduction to **Server Automation**

What automation is, why it matters in server environments, and how tools like Ansible let you manage systems at scale without doing the same task twice.

THE PROBLEM WITH DOING THINGS BY HAND

Imagine your organization runs twenty Linux web servers. Each one needs the same configuration: a specific package installed, a config file in the right location, a service running, and a firewall rule. On one server — ten minutes. On twenty servers — three-plus hours. And that assumes no typos, no missed steps, and no changes needed next week.

MANUAL ADMINISTRATION

- SSH into each server individually
- Run the same commands over and over
- Hope nothing was missed or mistyped
- No record of what was changed or when
- Inconsistency builds up over time
- Adding a server means starting over

AUTOMATED ADMINISTRATION

- Write the task once as a playbook
- Run it against one server or one hundred
- Every server gets identical results
- Every run is logged and repeatable
- Configuration is consistent by design
- Adding a server means adding one line

WHAT SERVER AUTOMATION ACTUALLY MEANS

Automation means describing what a system should look like — its packages, files, services, users, and settings — and letting a tool apply that description consistently across any number of servers.

CORE CONCEPT: DESIRED STATE

Rather than scripting every action, automation tools work by defining what the end state should be. The tool checks the current state, compares it to the desired state, and only makes changes where there is a difference. If the desired state already exists — nothing happens.

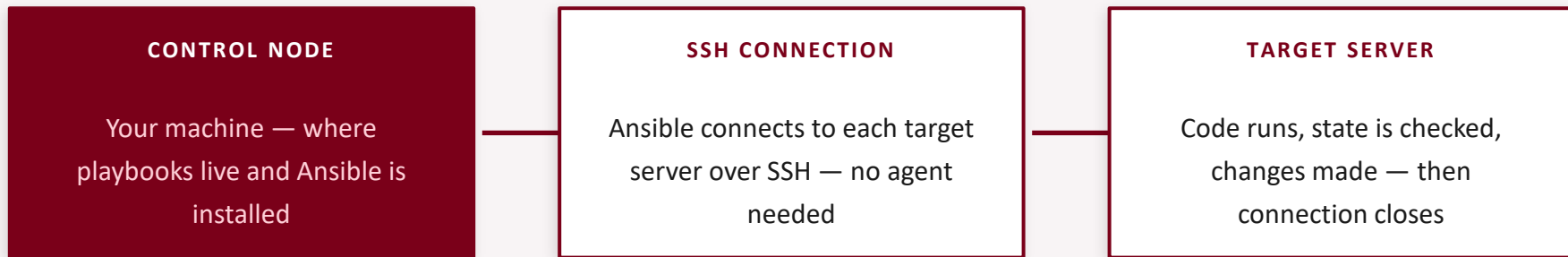
What Kinds of Tasks Get Automated?

- Package Management — Ensure software is installed — or not installed — across all servers
- Service Management — Start, stop, enable, or restart services as needed, consistently
- System Hardening — Apply a security baseline to every new server before production
- File Deployment — Push config files to the correct locations with the correct permissions
- User Management — Create accounts, set permissions, manage SSH keys at scale
- Backups & Compliance — Verify systems are in the expected state on a schedule

WHAT IS ANSIBLE?

Ansible is an open-source automation tool that lets you manage servers, apply configurations, deploy software, and orchestrate complex tasks — all from a single control node. No agents on target machines. No custom language. Just SSH and Python.

How Ansible Works



No software remains on the target server after the job is complete. Ansible makes its changes and disconnects.

ANSIBLE: KEY COMPONENTS

Component	What It Is	Analogy
Control Node	The machine where Ansible is installed and playbooks are run from	<i>The engineer's workstation</i>
Inventory	A file listing the servers Ansible can manage, organized into groups	<i>An address book</i>
Playbook	A YAML file describing what should be done and on which servers	<i>A work order</i>
Task	A single unit of work within a playbook — one thing to check or change	<i>A line item on the work order</i>
Module	A built-in tool that knows how to perform a specific type of action	<i>The right tool for the right job</i>
ansible.cfg	The project config file — tells Ansible where to find the inventory	<i>Project settings</i>

WHAT A PLAYBOOK LOOKS LIKE

Playbooks are written in YAML — a human-readable format using indentation to define structure.

Pattern:

- A play targets a group of servers
- Inside the play are tasks
- Each task calls a module to do one thing
- Desired state, not commands

Notice state: present and state: started — these are desired state declarations, not commands. If Apache is already installed, Ansible does nothing. The outcome is the same regardless of the starting point.

deploy_webserver.yml

```
---
- name: Configure web servers
  hosts: webserver
  become: yes
  tasks:

    - name: Ensure Apache is installed
      ansible.builtin.package:
        name: apache2 state: present

    - name: Deploy site config file
      ansible.builtin.copy:
        src: files/site.conf
        dest: /etc/apache2/...

    - name: Ensure Apache running
      ansible.builtin.service:
        state: started enabled: yes
```

IDEMPOTENCY — THE MOST IMPORTANT CONCEPT

DEFINITION

Idempotent: An operation that produces the same outcome regardless of how many times it is executed. The first run may make changes. Every subsequent run confirms those changes are still in place — and makes no new changes if they are.

What Ansible's Output Tells You

ok

The desired state already existed — nothing changed.

System was already correct. No action taken.

changed

The desired state did not exist — Ansible made a change.

Something was different. Ansible fixed it.

failed

The task could not complete.

Something went wrong. Check the error output.

A playbook that reports all ok on its second run is a well-written playbook — automation working exactly as intended.

WHERE THIS FITS IN YOUR PROGRAM

This lab is intentionally simple — you are running Ansible against your own VM on localhost. The goal is to learn the mechanics: installation, project structure, what a playbook looks like, and what idempotency feels like in practice.



Why This Skill Matters in the Industry

Manual server administration at scale is no longer viable. Employers hiring for server and systems roles increasingly assume familiarity with Ansible, Terraform, or similar platforms. Starting here — in your second semester — puts you ahead of that curve.

REVIEW QUESTIONS

Answer these before starting the lab.

- 1 What is the difference between a shell script that runs commands and an Ansible playbook that declares desired state? Why does that difference matter?
- 2 What does the Ansible inventory file do? If you had fifteen servers to manage, what would change in the inventory compared to managing just one?
- 3 A task shows ok on the second run. Your classmate says the playbook is broken because nothing happened. How do you explain why this is correct behavior?
- 4 Without running it, describe what state the target server will be in after the sample playbook runs successfully. Be specific.
- 5 Why does Ansible not require software to be installed on the servers it manages? What does it use instead?

Lesson 7

Summary

